

uringscope: Portable, Low-Overhead Observability for io_uring

Rajarshi Chowdhury
orcid.org/0009-0007-7032-2450

Abstract

io_uring moves I/O submission and completion into shared-memory rings. This makes it fast, and it also makes it invisible. **strace** sees only the ring setup, and the kernel tracepoints that expose the request flow are not stable ABI, so the few tools built on them work only on narrow kernel ranges. We present URINGSOPE, a single-binary, language-agnostic observability tool for io_uring built on CO-RE (Compile Once, Run Everywhere) eBPF. URINGSOPE makes four contributions. The first is a precise model of the request lifecycle and a method to reconstruct per-request flows from kernel events. The second is a technique for attaching portably to an unstable tracepoint surface, using BTF-probed program variants, CO-RE field flavors, and position-independent reads. The third is an evaluation of the trade-off between overhead and fidelity: on device-bound NVMe workloads URINGSOPE’s aggregate mode costs 0.7 to 9.9% of throughput, which is cheaper than every full-fidelity alternative we measured. The fourth is a lightweight correctness mode that reuses the same reconstruction to detect submission-boundary hazards, together with a built-in *doctor* that turns the measurements into named pathologies with evidence, for operators who are debugging a tail-latency incident rather than browsing histograms.¹

1 Introduction

For two decades, the answer to “what is my process doing to the kernel?” has started with **strace**. io_uring breaks that contract by design. Submission queue entries (SQEs) and completion queue entries (CQEs) travel through memory-mapped rings, and a busy application may issue millions of I/O requests through a handful of **io_uring_enter** calls, or none at all when it uses **SQPOLL**. Tracing the syscall boundary of such an application yields a stream of opaque **io_uring_enter** calls and none of the requests inside them. This limitation was raised on the strace mailing list in 2020 and remains unaddressed [11].

The information is not hidden. It is merely unpackaged. The kernel exposes roughly eighteen static tracepoints covering the io_uring request lifecycle [7], plus ring state in `/proc/<pid>/fdinfo`. Practitioners who need answers today are pointed at raw **tracefs**, **perf**, and hand-written **bpfftrace** scripts [8]. The one packaged tool, **uring-trace** [10], demonstrated the demand but also the obstacle. Its authors describe the difficulty of building a stable tool on top of unstable interfaces, and it supports only kernels 6.1 through 6.7. io_uring’s tracepoints are explicitly

not kernel ABI. They have been renamed, re-prototyped, and re-fielded repeatedly across the 5.15 to 6.x span (§5).

This paper argues that the gap is closable, and that closing it surfaces two problems of independent interest. The first is a semantic problem. Mapping kernel events back to application-meaningful request flows requires a precise model of the io_uring request lifecycle: when requests complete inline, when they park on the poll-retry path, when they silently fall back to the **io-wq** asynchronous worker pool (a classic hidden source of tail latency), and how multishot and linked requests break the one-submit-one-completion assumption. To our knowledge this model exists only as kernel source. We make it explicit (§3) and build URINGSOPE’s reconstruction on it. The second is a mechanical problem: attaching to a tracepoint surface that changes across kernel versions. CO-RE relocations solve struct-layout drift but not renamed or re-prototyped tracepoints. §5 contributes a layered technique that combines struct-centric reads, BTF-probed program variants, CO-RE flavors, and a kernel-matrix CI that treats portability as a continuously tested claim. We position this as a constructive counterpart to DepSurf’s measurement of the eBPF dependency-surface problem [12].

URINGSOPE packages both into a single static binary, invoked as **uringscope ./myapp**. Its default mode aggregates per-opcode latency histograms and pathology counters entirely in kernel maps, which is cheap enough to leave on in production. A trace mode streams per-request lifecycle records into a Perfetto-compatible timeline [4]. A *doctor* layer converts the measurements into verdicts with evidence and suggested fixes.

Contributions:

- A request-lifecycle model for io_uring and a per-request flow reconstruction that handles multishot operations, linked SQEs, **SQPOLL** submission, and deferred task-work delivery (§3).
- A portable-attachment technique for unstable tracepoint surfaces, with a churn study of io_uring’s tracepoints across 5.15–6.x (§5).
- URINGSOPE itself, open source, plus an overhead-vs-fidelity evaluation against **perf**, **bpfftrace**, and **strace** at up to saturation IOPS (§7).
- A lightweight *correctness* mode that reuses the lifecycle reconstruction to detect submission-boundary hazards. These are overlapping in-flight operations on a buffer, registered-buffer lifetime violations, dropped requests that are never reaped, and the unmap variant of buffer use-after-free. We give an explicit account of which hazards are and are not detectable from the kernel side (§6).

¹Source code and the evaluation artifacts that reproduce this paper’s figures and tables are archived at <https://doi.org/10.5281/zenodo.20672341>.

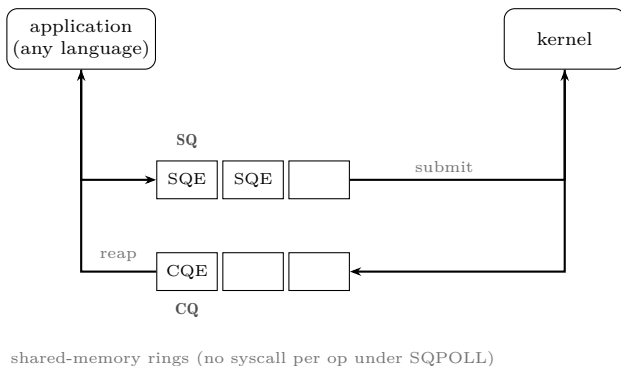


Figure 1: `io_uring`'s shared rings. The application writes SQEs to the submission queue and reads CQEs from the completion queue. The request flow lives in this shared memory.

2 Background and Motivation

2.1 `io_uring` in one column

`io_uring` [1] replaces the synchronous syscall boundary with two memory-mapped rings shared between the application and the kernel. The application fills a submission queue (SQ) with submission queue entries (SQEs), and the kernel fills a completion queue (CQ) with completion queue entries (CQEs). One `io_uring_enter` syscall can submit a batch of SQEs and reap many CQEs. With `SQPOLL`, a kernel thread polls the SQ and the application issues no syscalls at all. The libc-level wrapper `liburing` hides the ring bookkeeping (Figure 1). Four features matter for observation. *Registered* files and buffers are referenced by index rather than by fd or pointer. *Multishot* operations post many CQEs for one SQE. *Linked* SQEs form ordered chains. And `DEFER_TASKRUN` (6.1 and later) batches completion delivery into ring-local task work. Each of these changes how a request flows from submission to reaping, which is what an observer must reconstruct (§3).

2.2 Why existing tools fail

`strace` decodes the arguments to `io_uring_setup` and `io_uring_register`, but the request flow lives in the shared rings, which it never inspects. The 2020 `strace-devel` thread [11] that raised this produced no fix. Our measurements (§7) show that its syscall-stop machinery adds 44.4 μ s per request and distorts p99.9 latency by 33 $\times$, so it changes the behavior it is meant to observe.

`perf record -e 'io_uring:*` and hand-written `bpfttrace` scripts read the same tracepoints `URINGSCOPE` does, and for counting they work. But they have three problems. They emit raw events with no request-level correlation, so a `queue_async_work` event is just a name, not “this READ for this `user_data` took the slow path.” They defer all semantics to offline processing. And they collapse at high IOPS because every event crosses to userspace (§7.3), and break across kernels when a prototype changes (§5). The kernel also exposes cheap complementary

state in `/proc/<pid>/fdinfo`. `URINGSCOPE` parses it for ring geometry and `SQPOLL` identification (the SQ and CQ masks and the `SQPOLL` thread id); instantaneous queue depth instead comes from a CO-RE read of the ring’s completion-queue head and tail (§5.1), and io-wq worker counts from a BPF map fed by `sched_switch`. `fdinfo` is real and useful, but it is a snapshot, not a request history.

The one packaged tool, `uring-trace` [10], is closest in spirit, since it also uses eBPF with Peretto output, and it demonstrates the demand. But it is per-event by design and supports only kernels 6.1 through 6.7, which is the exact brittleness §5 addresses.

2.3 Who needs this

The audience is specific and worth stating honestly. `io_uring` is blocked by Docker’s default seccomp profile [3] and was disabled fleet-wide at Google for attack-surface reasons [6]. The relevant deployments are therefore bare-metal and VM storage engines, databases, and `io_uring`-native runtimes such as `fio`, TigerBeetle-class systems, and the `tokio-uring`, `glommio`, and `eio` ecosystems. This is a small population with a high blast radius. These are precisely the systems where a hidden async-worker punt becomes a customer-visible tail-latency regression, and where the engineer debugging it today has no tool between `strace`, which is useless here, and reading kernel source.

3 The `io_uring` Request Lifecycle

A request that an application thinks of as “one read” takes one of several kernel paths, and which path it took is the whole story for tail latency. We make the model explicit, and `URINGSCOPE`’s reconstruction is built on it.

Figure 2 is the model. After submission, the kernel issues an SQE on one of three paths. On the inline path it completes immediately, which is the fast path. On the poll-armed path it is pollable but not ready, so it is parked and reissued. On the punt path it must block and cannot poll, as with buffered I/O that misses the cache or an `fsync`, so it goes to the io-wq worker pool. The punt is the classic hidden tail-latency source, invisible to an application that sees only a slower completion. Completion then posts a CQE, takes an overflow slow path if the CQ is full, and is delivered via task work, batched under `DEFER_TASKRUN`. Each transition is exposed by exactly one tracepoint, named on the corresponding edge.

Reconstructing per-request flow requires a correlation key. On modern kernels the `io_kiocb` pointer is stable from submit to completion and is unique. Pre-5.19 legacy tracepoints force the weaker `(ctx, user_data)` pair, and `user_data` is application-chosen and freely reused, so it cannot be a sole key. Three semantics complicate the simple submit-to-complete mapping. *Multishot* operations post many CQEs per SQE, flagged `IORING_CQE_F_MORE`, so `URINGSCOPE` re-arms its in-flight record rather than retiring it, and latency means the inter-completion gap. *Linked* SQEs do not start until their predecessor completes, so their submit-to-complete latency legitimately includes queue-behind-link

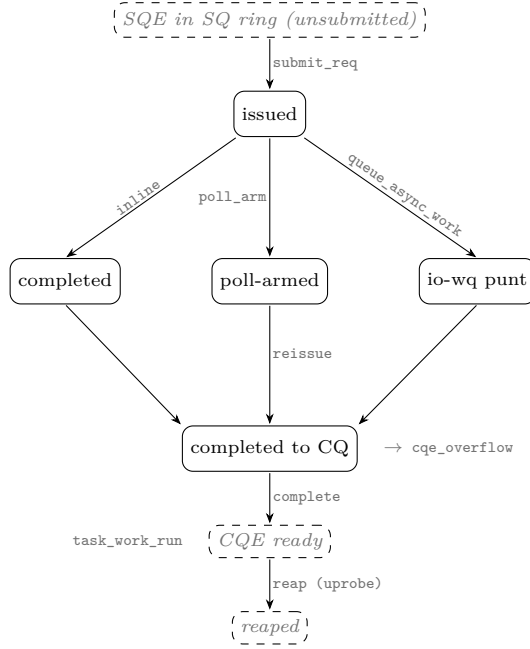


Figure 2: io_uring request lifecycle. Solid nodes are kernel states with the tracepoint that makes each transition observable. Dashed nodes are userspace-only segments that fire no tracepoint, reachable only via `liburing` uprobes. The three issue paths, inline, poll-armed, and io-wq punt, are the fork that determines tail latency.

time. `SQPoll` performs submission on the `iou-sqp` kernel thread, not in the application’s syscall context. This is why `URINGSOPE` filters by ring ownership, captured at `io_uring_create`, rather than by the pid observed at the tracepoint.

Two segments are fundamentally invisible to kernel tracepoints. One is the gap between an application preparing an SQE and submitting it. The other is the gap between a CQE becoming ready and the application reaping it. This second gap, completion-reaping lag, is a real source of perceived latency. `URINGSOPE` detects it from kernel-side completion-queue state, by spotting CQEs that sit ready while unconsumed. Measuring the precise per-reap delay needs optional `liburing` uprobes, which we discuss in §9.

4 uringscope Design

4.1 Two modes, one dichotomy

`URINGSOPE` offers two modes that trade fidelity for cost (Figure 3), and the choice between them is the backbone of the evaluation (§7). *Aggregate* mode folds everything into BPF maps in the kernel. These hold per-opcode log2 latency histograms, the pathology counters for Table 1’s signals, `SQPoll` off-CPU time, and io-wq fan-out. Userspace reads the maps once at exit, or periodically for Prometheus export. Nothing per-event crosses the boundary, so this is the

signal	granularity	rate @ 1M IOPS
<code>submit_req</code>	per request	1M/s
<code>complete</code>	per request	1M/s
<code>queue_async_work</code>	per punt	0–1M/s
<code>poll_arm</code>	per not-ready	network-only
<code>task_work_run</code>	per batch	10–100k/s
<code>enter syscall</code>	per batch	batching-dependent
<code>sched_switch</code>	system-wide	filtered in-probe

Table 1: Event rates per lifecycle signal. The two unavoidable per-request tracepoints alone produce 2M events/s at 1M IOPS. Shipping that to userspace is itself a workload, which is why `uringscope`’s default mode aggregates in the kernel (§4).

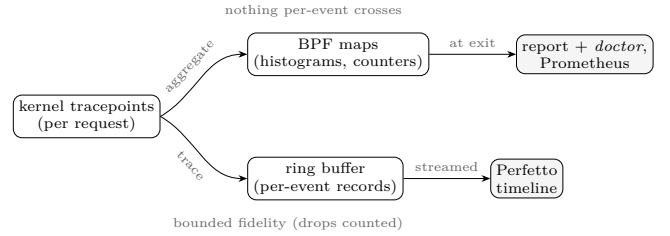


Figure 3: Two modes from one tracepoint set. Aggregate mode keeps per-event data in kernel maps, for production use. Trace mode streams records to a timeline, for diagnosis. The split between in-kernel aggregation and bounded-fidelity streaming is what the evaluation measures.

always-on, production-candidate mode. *Trace* mode instead streams one per-request record over a ring buffer to a Perfetto timeline. Its fidelity is bounded by ring-buffer capacity, and `URINGSOPE` reports any drops as a first-class counter rather than losing events silently. Reconstruction fidelity was 100% in both modes across our grid (§7).

4.2 Correlation and state

The hot structure is an in-flight hash keyed by the `io_kiocb` pointer (§3). It stores a 48-byte record per outstanding request: the submit and punt timestamps, opcode, `user_data`, flags, the owning `tgid`, and the target descriptor (address, length, buffer index, and kind) consumed by the `-check` hazard tier (§6). Entries are deleted on completion. Some completions find no matching entry, because the request was submitted before attach or evicted under pressure, and these are counted as untracked. That count is a first-class fidelity metric the report surfaces rather than hides. A separate ring-ownership map, populated at `io_uring_create` and back-filled from `/proc/<pid>/fd` when attaching to an already-running process, lets `URINGSOPE` filter to a target ring correctly even under `SQPoll`, where submission happens on a kernel thread. `SQPoll` and io-wq worker accounting reads `sched_switch` and filters on the `iou-sqp-` and `iou-wrk-` comm prefixes inside the probe, so that only the relevant context switches reach a map.

change	kernel	URINGSCOPE's response
submit_sqe → submit_req rename	5.18–6.0	two program variants, BTF-probed
io_kiocb.user_data into embedded io_cqe	5.19	CO-RE flavor + field_exists
cqe_overflow added	5.19	probe-by-name, absent = off
short_write added	6.0	probe-by-name
local_work_run added	6.1	probe-by-name
complete args → io_uring_cqe *	6.17	third variant, read via cqe

Table 2: io_uring tracepoint churn handled by uringscope, with the mechanism for each. The 6.17 complete collapse was encountered live on the measurement host (§7). Format dumps from each kernel are the table’s primary sources.

4.3 The doctor

On top of the aggregates sits a *doctor*, a set of conservative predicates that each name a known pathology, show the supporting evidence, and suggest a fix. The shipped rules cover the per-opcode punt ratio (buffered-I/O storms), submission batching (SQEs per io_uring_enter), SQPoll off-CPU fraction, unbounded io-wq fan-out, CQ overflow, short writes, deferred task-work delivery (DEFER-TW), per-opcode error rate, and dropped requests that are never reaped, plus tool-fidelity self-reports. The governing principle is that a checker which cries wolf is ignored, so thresholds favor precision over recall. The evaluation validates this stance with explicit false-positive guards (§7.10). The report, findings included, is also emitted as machine-readable JSON under a versioned schema, and a `-fail-on` severity threshold maps findings to deterministic exit codes, so the doctor can gate CI runs and agent-driven workflows. The doctor is the feature that turns “writes are slow” into “fsync is punting 100% to the bounded worker pool and owns your p99” (§7, case study).

5 Surviving the Unstable Dependency Surface

Kernel tracepoints are explicitly not ABI, and io_uring’s have exercised that freedom repeatedly. Table 2 catalogs the changes URINGSOPE handles across 5.15 to 6.17. Dep-Surf [12] formalizes this instability as a “dependency surface,” and this section is the constructive counterpart, a tool engineered to survive it. CO-RE relocates struct field offsets automatically, but it does nothing for renamed or re-prototyped tracepoints, which need the layered technique below.

Then the technique, layered by what changed:

- **Fields moved** ⇒ CO-RE flavors + `bpf_core_field_exists()` (one compiled program serves both layouts).
- **Prototypes reshuffled** ⇒ struct-centric reads: trust the `io_kiocb` pointer, distrust argument positions.
- **Tracepoints renamed/added/removed** ⇒ multiple compiled variants. Userspace probes kernel BTF for `btf_trace_*` symbols and flips autoloader per variant,

so a missing tracepoint degrades one feature instead of failing load.

- **The future** ⇒ a nightly kernel-matrix CI (5.15, 6.1, 6.6, 6.8, 6.12, and mainline, the last currently supplying 6.17 coverage) that boots each kernel, runs a known workload, asserts the expected support tier, and archives tracepoint formats, making portability a regression-tested property.

Concretely, the portability machinery is a small fraction of the tool. The BPF object defines 18 kernel-attached programs (plus two liburing uprobes for the userspace boundary, 20 in all) across 1,340 lines, and the variant-selection logic in userspace is 412 lines. Four of the programs are variants of a single tracepoint, `io_uring_complete`: the modern 5-argument form, the pre-6.0 legacy form, the 6.17 cqe-collapsed form, and a count-only fallback. Two more are submit-path variants, `submit_req` versus the legacy `submit_sqe`, and the remaining programs attach once. At load time, nine autoloader decisions driven by BTF probing select the variants appropriate to the running kernel and disable the rest, and two further hazard hooks are enabled by tracefs probing, for eleven rows in the startup support summary. The cost of spanning 5.15 to 6.17 is thus a handful of alternative programs and a probe table, not a rewrite per kernel.

5.1 Empirical validation of the technique

The mechanism is not hypothetical. On a stock 6.6 kernel, libbpf load-time logs confirm each layer firing as designed. The 5.19 field move, in which `io_kiocb`’s `user_data` migrated into an embedded `io_cqe`, is handled live. The modern relocation resolves `io_kiocb.cqe.user_data` to its kernel offset, while the `io_kiocb__pre519` flavor finds no match and its load is poisoned by `bpf_core_field_exists()`, which is exactly the intended degrade-not-fail behavior. The struct-centric instantaneous-depth read relocates `io_ring_ctx.rings` to `io_rings.cq.{head,tail}` without touching tracepoint arguments. BTF probing then disables the legacy `submit_sqe` and `complete` variants on this kernel and enables the modern `submit_req` path. The startup support summary on the measurement kernel makes the selection legible. It reports each feature as active with the variant chosen, for example `completion: active, v6.17 cqe-collapsed (3-arg)` and `submission: active, modern submit_req`, and it disables the legacy variants. Table 2 lists the per-change mechanism, and the full multi-kernel coverage matrix is left to the CI described in §9.

6 Correctness: Hazards at the Submission Boundary

io_uring inverts the ownership contract of synchronous I/O. A buffer handed to the kernel at submit is not the application’s again until the matching completion. Four hazards live in that window. The first is use-after-free or `munmap` of an in-flight buffer. The second is two in-flight operations writing

hazard	vantage	status
dropped requests (4)	kernel tracepoints	shipped
overlapping in-flight (2)	kernel tracepoints	shipped
registered-buf lifetime (3)	+ register tp	shipped
UAF, munmap variant (1)	+ munmap tp	shipped
UAF, free/stack variant (1)	needs allocator hooks	disclaimed
reaping lag (CQEs ready)	kernel tracepoints	shipped

Table 3: Submission-boundary hazards by detectability. The honest line between what kernel-side tracing can and cannot catch is itself a result.

the same buffer range, which silently corrupts data and never returns an error. The third is a registered-buffer lifetime violation, where an index is unregistered or re-registered while it has live references. The fourth is a dropped request that is submitted but never reaped, which pins the buffer forever. The same lifecycle tracking that powers observation makes a lightweight correctness checker possible, a “valgrind for the io_uring submission boundary.”

The contribution here is as much epistemic as mechanical. It is a precise statement of which hazards are detectable from which vantage point (Table 3). Hazards 2, 3, and 4, together with the `munmap` variant of hazard 1, are reachable from kernel tracepoints alone, because URINGSCOPE already tracks every in-flight request keyed by `io_kiocb`. The freelist or stack-reuse variant of hazard 1 fires no syscall and is invisible without allocator instrumentation. We disclaim it explicitly rather than overclaim.

URINGSCOPE’s `-check` mode implements these as a debugging tier, an “ASan for the io_uring submission boundary” rather than the always-on path. It maintains per-ring in-flight target ranges, using address and length for plain operations and `buf_index` and offset for fixed operations, captured at submit by CO-RE-reading the `io_kiocb`. From these it runs four checks. The first is an overlap test on submit (hazard 2). The second validates `buf_index` reference counts against the register tracepoint (hazard 3). The third tests `munmap` ranges against in-flight targets (hazard 1, unmap variant). The fourth is an aged in-flight scan for dropped requests (hazard 4, always available). All four are validated by injection in §7.10.

The BPF verifier shapes this design. The overlap test cannot iterate the in-flight hash map, because the verifier rejects unbounded loops. URINGSCOPE therefore keeps a fixed-size per-ring scan window of the K most recent in-flight target descriptors and tests new submissions against it with a `#pragma unrolled` comparison. We set K to 64. Hazard-2 detection is thus a bounded approximation. An overlap against an in-flight request older than the last K on the same ring is not caught by this rule, although the dropped-request rule (hazard 4) still flags the very old stragglers. We state this limit explicitly rather than imply exhaustiveness. For a debugging-mode checker, last- K coverage with a documented bound is the honest and verifier-feasible point. A hazard is surfaced via an at-exit counter-plus-samples map, the same transport as the dropped-request rule. It carries

both colliding `user_data` tokens, both opcodes, and the overlapping range, so a developer can grep their own code for the two requests involved. In our implementation the in-kernel bounded scan held at $K = 64$ and passed the verifier on Linux 6.17. All three kernel-side hazard rules, namely overlapping in-flight, registered-buffer lifetime, and the `munmap` variant of use-after-free, detect their injected hazards with no false positives (§7.10). On the overlap injection the doctor reports, verbatim, “`READ_FIXED(user_data=0x41)` and `READ_FIXED(user_data=0x42)` overlap in registered buffer #0 at `[...,+4096]`”, alongside `[HAZARD-BUFREG]` flagging the subsequent unregister of an index with two live references. The two detectors fire together on one workload.

Detection effectiveness is scored by the injection harness of §7.10. This positions URINGSCOPE distinctly from RingGuard [5], which applies io_uring+eBPF to security *policy enforcement*. Correctness checking of the buffer-ownership contract is a separate and, to our knowledge, unoccupied point.

7 Evaluation

7.1 Implementation status

Before the quantitative results, we state what is built and validated, so that claims and measurements are not conflated. All eleven injection-validated doctor rules detect their injected pathology on the measurement kernel (Linux 6.17.0-1017), and the false-positive guards hold, as detailed in §7.10. The shipped rule set is larger than the injection-validated eleven (§4). Detection effectiveness is a correctness property, validated independently of the performance campaign. The performance results below come from a 36-cell grid of 6 workloads by 6 observers, with 5 runs per cell and medians reported, collected on the host described in §7.2.

7.2 Experimental setup

The protocol is implemented in `bench/`. It is a grid over workloads, observers, and load levels, with at least 5 runs per cell, fio pinned to a fixed core set, and cold-cache control via `drop_caches`. The observers are nothing (baseline), URINGSCOPE-aggregate, URINGSCOPE-trace, `perf record -e 'io_uring:*'`, a `bpftrace` counter one-liner, and `strace -c`. The workloads are the `O_DIRECT` fast path, a buffered cold-cache punt storm, an `fsync` writer, `SQPoll`, `iodepth-1`, and a network echo server. The five storage workloads are fio-driven; the network echo cell is instead driven by `nc` echo streams against the server.

We run on a single host: Intel Xeon Platinum 8375C (Ice Lake, 2.9 GHz), 32 logical CPUs (16 cores $\times$ 2 SMT), 256 GiB RAM, Ubuntu 24.04, `CONFIG_DEBUG_INFO_BT=y`. The CPU-bound (`null_blk`) workloads ran on Linux 6.17.0-1017, and the device-bound NVMe workloads on 6.17.0-1012, an earlier point release of the same series. The AWS kernel advanced mid-campaign, which is itself a minor illustration of the moving target §5 addresses. The physical storage device is a datacenter NVMe SSD whose ceiling we report from measurement rather than a part number: 322k

4 KiB random-read IOPS at QD64, p99 completion latency 232 μ s, p99.99 255 μ s, at 99.5% device utilization. Two honesty notes. First, the upper-bound overhead numbers are measured against an in-kernel `null_blk` device: this removes storage from the critical path and makes the workload CPU-bound, so observer cost translates directly into throughput loss. This is a deliberate worst case. The physical NVMe carries the workloads where real-device semantics matter (cold-cache reads, fsync) and represents the device-bound production regime. Second, the host is a virtualized instance (thin hypervisor): C-states and turbo are not pinned by the guest. We mitigate by anchoring the worst-case claim on `null_blk` (largely jitter-independent), reporting medians of 5 runs per cell, and noting a bare-metal re-run as future work. Each cell is a 60s run after 10s warmup, fio pinned to 4 cores, observers unpinned on the remaining 28.

7.3 Overhead

Figure 4 shows throughput loss per observer, and Figure 5 restates the CPU-bound case as cost per request, which is the durable number. There are three results.

First, device-bound workloads, which are the production case, are cheap to observe. On the NVMe `O_DIRECT` fast path (330k IOPS), URINGSOPE-aggregate costs 2.2%, on write+fsync it costs 0.7%, and on buffered cold-cache reads it costs 9.9%. It is cheaper than `perf record -e 'io_uring:*`' on all three (2.3%, 7.0%, and 13.3%), while delivering reconstructed per-request state rather than a raw event file that needs post-processing. The reason is headroom. When the device is the bottleneck, tracepoint cost is absorbed into CPU idle time instead of displacing I/O work.

Second, at CPU-bound saturation, in-kernel aggregation is not free, and we report that plainly. Against `null_blk` at 810k IOPS, URINGSOPE-aggregate costs 31.9% of throughput, or 583ns per request. That is the price of an in-flight hash insert and delete plus a histogram update on every submission and completion pair. A count-only `bpftime` probe costs 42ns (3.3%) but answers no question beyond “how many.” `perf` costs 837ns (40.2%) and defers all semantics to offline processing, and URINGSOPE-trace costs 1,221ns (49.5%). The naive claim that in-kernel aggregation makes observation free at any load is false, and our data says so. The defensible claim is that cost per unit of fidelity is the right metric, and on that metric URINGSOPE-aggregate dominates the alternatives that can actually answer latency-attribution questions.

Third, where the cost lands also differs. Whole-system CPU accounting shows URINGSOPE-aggregate within run-to-run noise of baseline (7063 versus 7050 jiffies), because its work runs inline in the traced task and is already counted in that task’s time. By contrast, `perf` and URINGSOPE-trace roughly double system CPU (13.8k and 15.1k jiffies), since each burns an additional core on a userspace collector. On a machine with spare cores this is invisible. On a saturated one, per-event collectors steal a core from the workload’s neighbors. `strace` is the anti-baseline at 44.4 μ s per request and

93 to 97% throughput loss. The one exception is `SQPOLL`, where it costs only 0.2% because there are no syscalls to intercept. Its overhead reaches zero exactly when its information content does.

7.4 Fidelity

Across the grid, URINGSOPE reconstructed 100% of completed requests in both modes: in the heaviest cell, 36,838,423 submissions against 36,838,148 completions with 275 legitimately in flight at detach, zero untracked completions, and zero ring-buffer drops as counted by trace mode’s drop counter (`trace_rb_drops`), which the report surfaces directly. Trace mode sustained full per-request capture at 420k IOPS (28.6M requests in one 68s cell). The fidelity comparison with the alternatives is categorical rather than numeric. The count-only probe reconstructs nothing. `perf` captures raw events whose request-level correlation is deferred to offline tooling. And `strace` cannot see the request flow at all.

7.5 Tail perturbation

Figure 6 shows the tail. On device-bound NVMe workloads every eBPF-based observer holds p99.9 within 1.06 $\times$ of baseline, and URINGSOPE-aggregate stays within 1.02 to 1.03 $\times$. At CPU-bound saturation URINGSOPE-aggregate reaches 1.42 $\times$ and `perf` reaches 1.59 $\times$. `strace` distorts p99.9 by 33 $\times$, which disqualifies it from tail investigation even where its throughput cost might be tolerated.

7.6 Portability, exercised live

The portability mechanism was tested by events, not just by CI. The measurement host runs Linux 6.17, released after this tool was written, and 6.17 changed `io_uring_complete`’s prototype again. The scalar `user_data`, `res`, and `cflags` arguments collapsed into a single `struct io_uring_cqe *`. URINGSOPE’s probe detected the unrecognized 3-argument form and selected it at load time, after one added program variant of about 40 lines. The startup support matrix on this kernel reports all nine lifecycle features active, with completion served by the “v6.17 cqe-collapsed (3-arg)” variant. The same campaign also validated the 5.19 `io_kiocb` field-move flavor (poisoned on this kernel, as designed) and degrade-not-abort behavior. Tracepoint format dumps from the measurement kernel are archived as the churn table’s primary sources.

7.7 End-to-end boundary instrumentation

The two userspace-side lifecycle segments (§3) are reached with best-effort `liburing` uprobes. On the measurement host the tool located `liburing.so.2`, attached, and reported the submit-side gap directly. For a workload submitting through the library, it reported “1 `io_uring_submit()` call, avg 2.0 *SQEs pending/call*.” The reap-side gap, however, was not observable for our workloads, and we report this honestly. `liburing`’s completion-peek path is frequently inlined, emitting no library call for a uprobe to attach to, so reap-lag timing is unavailable unless the application reaps through

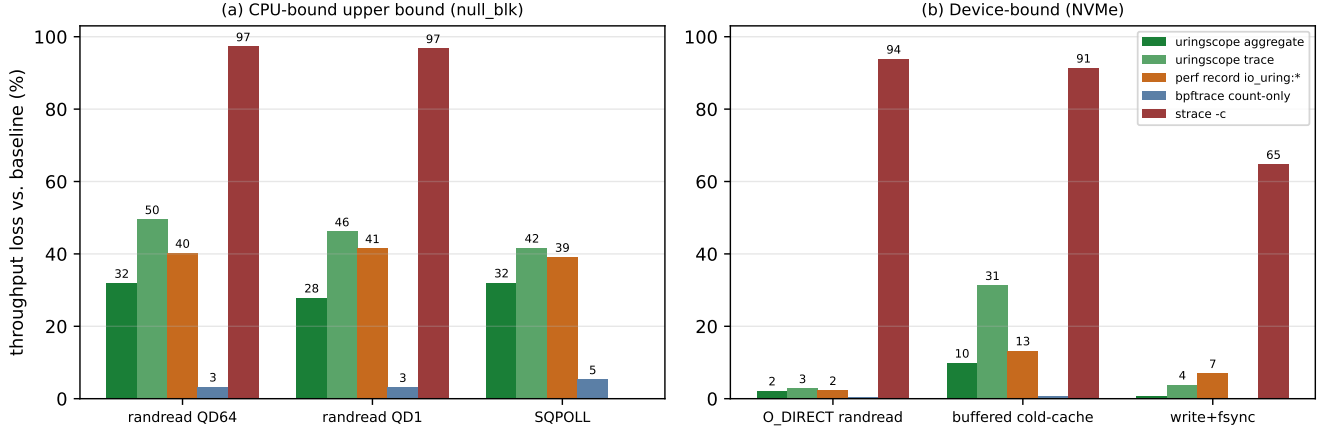


Figure 4: Throughput loss per observer. (a) CPU-bound null_blk workloads are the deliberate worst case, where every nanosecond of observer cost subtracts from throughput. (b) On the device-bound NVMe workloads that represent production, uringscope’s aggregate mode costs 0.7 to 9.9% and is the cheapest full-fidelity observer on every workload. Medians of 5 runs.

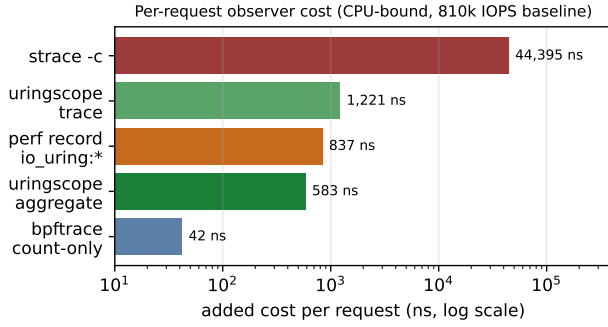


Figure 5: Per-request observer cost, derived from inverse-throughput deltas at the CPU-bound saturation point (810k IOPS baseline). This is the load-independent way to state overhead. Multiply by your IOPS to predict throughput cost when CPU-bound. When device-bound, idle headroom absorbs it.

a non-inlined entry point. This is the predicted asymmetry. Kernel-side CO-RE gives one-binary portability, while uprobe attachment depends on the target library’s symbols and inlining. The end-to-end features are therefore explicitly a best-effort tier that degrades to kernel-side-only with a clear message rather than failing. The [REAP-LAG] rule is nonetheless validated by injection (§7.10), and what varies by target is whether a uprobe site exists to feed it.

7.8 Pathologies are kernel-version-dependent

We also had an unplanned finding. The classic “buffered-read punt storm,” the io_uring pathology most documented by practitioners, does not reproduce on 6.17. Across 14.27M

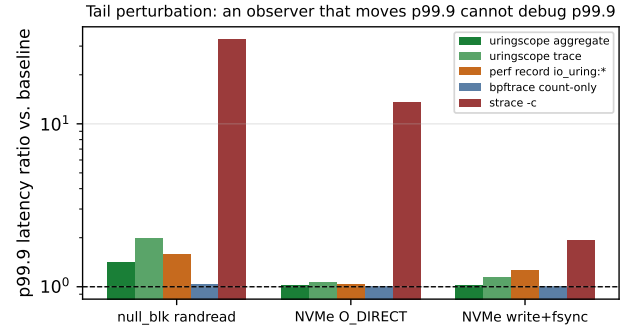


Figure 6: p99.9 latency under observation, relative to baseline (log scale). An observer that moves the tail cannot be used to debug the tail.

cold-cache buffered reads, 0.0% punted, because modern kernels retry buffered reads via page-lock waiting rather than io-wq. Buffered writes and fsync, by contrast, still punt 100% (3.03M ops in our fsync workload, attributed per-op as WRITE p50 262 μ s, FSYNC p50 1.0 ms, p99 4.2 ms). The pathology catalogue itself drifts with the kernel. This is an argument for a maintained measurement tool over folklore, and it is why URINGSOPE’s doctor reports evidence rather than assuming which pathologies a kernel can exhibit. The SQPOLL cells make the complementary point. We saw 1.46M IOPS with zero io_uring_enter() syscalls observed and the poller 0.0% idle, and the doctor correctly stays silent on a healthy saturated ring.

7.9 Case study: naming the pathology

The detection campaign supplies the case study directly. On the fsync workload the doctor attributes 100% of

injected pathology	doctor rule	result
io-wq punt storm	[WARN] punt ratio	detect
syscall-per-op	[INFO] batching	detect
CQ overflow	[CRIT] overflow	detect
error flood	[WARN] error rate	detect
dropped requests	[LEAK]	detect
SQPOLL stall	[WARN] sqpoll	detect
io-wq fan-out	[WARN] workers	detect
overlapping in-flight	[HAZARD]	detect
reg-buffer lifetime	[HAZARD-BUFREG]	detect
munmap in-flight	[HAZARD-UAF]	detect
reaping lag	[REAP-LAG]	detect
false-positive guards (clean workloads)		0 fired

Table 4: Detection effectiveness on Linux 6.17.0-1017: 11/11 injected pathologies detected, 0 false positives. Scored by the injection harness against machine-readable ground truth.

3.03M requests to io-wq punts and splits the latency by opcode (WRITE p50 262 μ s versus FSYNC p50 1.0ms, p99 4.2ms), turning “writes are slow” into “fsync detours through the worker pool and owns your tail.” On the iodepth-1 workload the doctor flags batching at 1.00 SQEs per `io_uring_enter()` across 28.6M calls. That is syscall-per-op, the very thing io_uring exists to avoid.

7.10 Detection effectiveness

The injection harness scores every injectable doctor rule against printed ground truth on the measurement kernel (Linux 6.17.0-1017). All 11 injection-validated rules detect their injected pathology with zero false positives: punt storm, batching failure, CQ overflow, error floods, dropped requests, SQPOLL stall, io-wq fan-out, overlapping in-flight buffers, registered-buffer-lifetime violation, `munmap-of-in-flight-buffer`, and reaping lag (Table 4). The false-positive guards hold. A no-batch workload does not trip the punt rule, routine worker counts do not trip the fan-out rule, and a clean workload reports no hazard. This includes the silent-corruption case, where two overlapping `READ_FIXEDs` into one registered buffer both return success (`res=4096`) while one writer’s data is destroyed. The doctor names both `user_data` tokens and the overlapping range so the developer can locate the two requests in their own code.

8 Related Work

The closest prior tool is `uring-trace` [10], which also uses eBPF with Perfetto output. It is per-event by design and pinned to kernels 6.1 through 6.7. Practitioner guides [7, 8] document manual tracepoint analysis, and the `strace-devel` thread [11] records the gap without closing it.

On portability, DepSurf [12] measures the eBPF dependency-surface instability that \$5 engineers around, and CO-RE [9] provides the field-offset half of the solution. RingGuard [5] combines io_uring with eBPF, but for security policy enforcement rather than observability or correctness.

More generally, `bcc/libbpf-tools` and `bpfftrace` [2] provide the substrate that URINGSOPE builds on. URINGSOPE differs in being io_uring-specific, semantics-aware, and packaged as a single binary, and it uses Perfetto [4] as its trace-mode output target.

9 Limitations and Future Work

URINGSOPE has real limits. The two userspace-side lifecycle segments (SQE-prep to submit, and CQE-ready to reap) are observed with best-effort `liburing` uprobes, and their visibility depends on how the application reaps (§7.7). The submit side is observable when submission goes through the library. The reap side is observable only when the application reaps through a non-inlined library call or a waiting `io_uring_enter`. Applications that inline the completion-peek path or read the completion ring directly expose no instrumentation point, so for them completion-reaping lag is unmeasured. The 5.15 legacy tier is counters-only, without per-request punt attribution. io_uring grows opcodes and semantics every cycle (zero-copy receive, futex operations, bundles), so the lifecycle model requires ongoing maintenance. The overlapping-in-flight check is bounded to the last K requests per ring. The seccomp and container reality (§2) limits the deployment surface by construction, and the measurement host being virtualized (§7.2) leaves a clean bare-metal re-run as future work.

10 Conclusion

The packaged-io_uring-observability slot stood empty not for lack of data, which the kernel has exposed via tracepoints for years, but because the surface was hostile: the request flow lives in shared memory, and the tracepoints are not ABI. A precise lifecycle model and a portability discipline of BTF-probed variants, CO-RE flavors, and struct-centric reads close the gap. The result is a single static binary that observes any language’s io_uring traffic on stock kernels from 5.15 through 6.17, costs 0.7–9.9% on device-bound workloads, reconstructs every request, names pathologies with evidence, and adapted to a kernel released after it was written. URINGSOPE is open source.

References

- [1] Jens Axboe. 2019. Efficient IO with io_uring. https://kernel.dk/io_uring.pdf.
- [2] bpfftrace contributors. 2026. bpfftrace: High-level tracing language for Linux. <https://github.com/bpfftrace/bpfftrace>.
- [3] Docker, Inc. 2026. Docker default seccomp profile. <https://docs.docker.com/engine/security/seccomp/>. `io_uring_enter/setup/register` are blocked by default.
- [4] Google. 2026. Perfetto: System profiling, app tracing and trace analysis. <https://perfetto.dev>.
- [5] Wanning He, Hongyi Lu, Fengwei Zhang, and Shuai Wang. 2023. RingGuard: Guard io_uring with eBPF. In *Proceedings of the 1st Workshop on eBPF and Kernel Extensions* (New York, NY, USA) (*eBPF '23*). Association for Computing Machinery, New York, NY, USA, 56–62. doi:10.1145/3609021.3609304
- [6] Tamás Koczka. 2023. Learnings from kCTF VRP’s 42 Linux kernel exploits submissions. Google Online Security Blog, <https://security.googleblog.com/2023/06/learnings-from-kctf-vrps-42-linux.html>.

- [7] Linux kernel contributors. 2026. Linux kernel io_uring tracepoints. `include/trace/events/io_uring.h`, Linux source tree. ~18 static tracepoints as of v6.12.
- [8] Marek Majkowski. 2022. Missing Manuals: io_uring worker pool. Cloudflare Blog, https://blog.cloudflare.com/missing-manuals-io_uring-worker-pool/.
- [9] Andrii Nakryiko. 2020. BPF CO-RE (Compile Once – Run Everywhere). <https://nakryiko.com/posts/bpf-portability-and-core/>.
- [10] OCaml Multicore contributors. 2024. uring-trace: Visualization tool for your io-uring workload. <https://github.com/ocaml-multicore/uring-trace>. Accessed June 2026. Supports kernels 6.1–6.7.
- [11] Miklos Szeredi. 2020. strace of io_uring events? strace-devel mailing list, <https://lists.strace.io/pipermail/strace-devel/2020-July/009933.html>.
- [12] Shawn Wanxiang Zhong, Jing Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2025. Revealing the Unstable Foundations of eBPF-Based Kernel Extensions. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (*EuroSys '25*). Association for Computing Machinery, New York, NY, USA, 21–41. doi:10.1145/3689031.3717497